

AKASH HADAGALI PERSETTI

Bloomington, IN | 812-837-7651 | hadagalipersettiakash@gmail.com
linkedin.com/in/akash-hp | github.com/akashpersetti | akashpersetti.com

SUMMARY

Applied AI engineer building reliable agentic systems, LLM evaluation infrastructure, and production AI applications on AWS. Built a Terraform-generating agent gated by real validation, a multi-provider evaluation platform with confidence intervals, and an open-source MCP server published to PyPI. Full-stack across LangGraph, FastAPI, Next.js, and serverless Terraform CI/CD on AWS. M.S. in Computer Science, Indiana University Bloomington (2026).

EXPERIENCE

AI Engineer (Remote)

MyEdMaster LLC

Jun 2026 – Present

Leesburg, VA

- Building an adaptive AI tutoring product that assesses a learner's existing knowledge of a target topic or certification and generates a customized, real-time training program covering only the gaps, using Python, LLMs (OpenAI SDK), and MySQL.
- Developed the initial assessment and curriculum-generation services using structured outputs, Pydantic validation schemas, and automated test cases, keeping generated coursework grounded and aligned to certification requirements while enabling rapid prototype iteration toward a QA-approved milestone.

Machine Learning Intern (Remote)

MyEdMaster LLC

Jun 2025 – Aug 2025

Leesburg, VA

- Built a real-time computer-vision exercise-assessment system that cut repetition miscounting by roughly 30% and reached over 92% count accuracy across 3 exercise types (Kettlebell Front Raise, Seated Leg Extensions, Jack Knives) by adding calibration routines and motion-consistency checks in Python and MediaPipe, lowering estimated coach intervention time by about 40%.
- Held form-detection latency under 100ms per frame by engineering real-time pose-landmark extraction and joint-angle calculations, delivering sub-second corrective feedback so users could self-correct during live sessions without instructor oversight.

PROJECTS

RepoModernizer | *Autonomous Codebase Migration Agent* | Fargate, EFS, Lambda repo-modernizer.akashpersetti.com

- Built a 5-node LangGraph state machine (ingest, install deps, plan, migrate-file loop, finalize) that plans a file-by-file migration from a natural-language goal, rewrites each file with an LLM, and gates any diff above a risk threshold behind a human-in-the-loop `interrupt()` before it touches disk.
- Detects a cloned target repo's own dependency manifest (pip or npm) and installs it into an isolated per-task virtual environment, decoupled from the worker's own runtime, so the target's real test suite runs against its real dependencies – and fails fast before spending any LLM cost if install itself breaks.
- Computed diffs from known-good before/after file content rather than trusting LLM-authored diffs, added bounded retries with Bedrock primary/fallback failover on transient or provider errors, and enforced guardrails (forbidden paths, max changed lines, no deletions) plus a hard per-task cost cap.
- Deployed on API Gateway, Lambda (FastAPI via Mangum), SQS, and one-shot Fargate tasks with DynamoDB checkpointing and an EFS-mounted workspace so a fresh container resumes mid-migration after a crash; provisioned through Terraform and GitHub Actions OIDC, with 96 passing tests and no long-lived AWS credentials.

TerraformAgent | *Multi-Agent IaC Generator* | LangGraph, FastAPI, Step Functions terraform-agent.akashpersetti.com

- Built a 6-node LangGraph pipeline (orchestrator, researcher, parallel domain subagents, aggregator, reviewer, evaluator) that turns a natural-language infrastructure request into validated Terraform, with a conditional edge that reruns only the domains a review flags for up to a capped number of targeted retries.
- Gated output on a real evaluator that writes the generated code to disk and runs `terraform fmt` and `terraform validate` against a provider cache baked into the Docker image, reaching full offline validation by copying the read-only cache into a writable path once per execution environment.
- Hardened the review step with a multi-LLM fan-out (reusing my mcp-second-opinion pattern) that queries OpenAI and Anthropic in parallel and blocks on any security veto, falling back to single-LLM review so a provider outage degrades quality instead of failing the run.
- Deployed the containerized service on Lambda and Step Functions with JWT authentication, DynamoDB state, SQS failure handling, and Secrets Manager; provisioned the stack through Terraform and GitHub Actions OIDC, with 94 passing tests and no long-lived AWS credentials.

EvalBench | *Multi-Provider LLM Eval Platform* | FastAPI, LiteLLM, Next.js, Terraform evalbench.akashpersetti.com

- Architected a pluggable-suite eval harness where every benchmark emits one shared **MetricRecord** contract persisted to a single JSON-column table, so adding a suite is a three-file change that never touches the runner, store, or dashboard core.
- Shipped 3 benchmark suites on that harness: structured-output reliability (schema-valid rate, retries-to-valid, retry-only cost accounting), latency/cost with pairwise judge scoring, and RAG with recall@k, nDCG@k, and faithfulness across swappable chunking strategies.
- Ran 27 evaluation runs across 75 tasks and 7 model/config variants, surfacing an 8-point first-attempt schema-validity gap between providers on structured output and a p95 latency spread of 6.0s vs 9.4s; found that semantic chunking doubled RAG context precision (0.13 to 0.26) over fixed-window at comparable faithfulness.
- Hand-implemented statistically honest aggregation, attaching 95% Wilson intervals to proportions and binomial order-statistic intervals to p95 latency, with per-metric sample sizes on every leaderboard cell so no aggregate renders as a bare mean.
- Deployed serverless on AWS: dual Lambda (API plus async runner), API Gateway v2, CloudFront, DynamoDB run tracking, and SES magic-link auth; backed by 262 passing tests using injected LiteLLM callables and temporary SQLite databases, running the full suite with no provider calls or model cost.

Wingman | *Self-Evaluating Agentic Co-Worker* | LangGraph, FastAPI, Lambda, DynamoDB wingman.akashpersetti.com

- Designed a worker-evaluator state machine in LangGraph where a structured-output evaluator checks worker responses against user-defined criteria and reinjects feedback for up to 5 retry turns, raising answer-validity rates before surfacing a result.
- Made the agent serverless-safe by reconstructing LangGraph state from DynamoDB on every request instead of an in-memory checkpoint, eliminating cold-start state loss across Lambda invocations.
- Integrated 5 tool domains through LangChain ToolNode and bind tools: web search (Serper), Wikipedia, sandboxed Python REPL, sandboxed file I/O, and Pushover notifications, and shipped the full serverless stack (containerized FastAPI on ECR, Lambda, API Gateway v2, CloudFront) via the same OIDC pipeline.

Twin | *AI Digital Twin + Live Eval Dashboard* | FastAPI, Next.js, AWS Bedrock, Lambda, S3 akashpersetti.com

- Built a tool-calling personal-website agent on Bedrock Claude Sonnet 4.5 using Titan-v2 retrieval over a persona corpus assembled at Lambda cold-start, with a pre-approved-FAQ tool and an escalate-to-human tool that SNS-notifies the owner for a live reply via a DynamoDB-backed admin inbox.
- Shipped a public evals dashboard scoring retrieval quality (recall@k, nDCG@k) on 35 labeled queries per push and LLM-judged faithfulness on live traffic via an S3-event-driven judge Lambda that adds zero latency to chat responses.
- Cut full-stack deployment to a single command across dev, test, and prod by chaining Docker build, Lambda upload, Terraform apply, Next.js export, S3 sync, and CloudFront invalidation through GitHub Actions and AWS OIDC.

mcp-second-opinion | *Open-Source MCP Server, Published to PyPI* | Python pypi.org/project/mcp-second-opinion

- Built and published an MIT-licensed MCP server that lets any MCP-aware agent consult rival LLMs (OpenAI, Gemini, Anthropic, Grok) mid-conversation, exposing two tools: one to query a single model and one to fan out to all enabled providers in parallel and compare answers.
- Unified four provider APIs behind a single LiteLLM interface, returning per-call latency, token counts, and cost; gracefully disables providers with no API key instead of failing, and ships as a pip-installable CLI (71 downloads in its first month) registered through standard MCP client config.

EDUCATION

Indiana University Bloomington

Graduated May 2026

M.S. in Computer Science, GPA 3.8/4.0

The National Institute of Engineering, Mysuru, India

2020 – 2024

B.Eng. in Computer Science and Engineering, GPA 3.7/4.0

TECHNICAL SKILLS

Languages: Python, TypeScript, JavaScript, SQL, C++

Agentic AI & LLM Engineering: LangGraph, LangChain, OpenAI SDK, Model Context Protocol (MCP, published server to PyPI), RAG, function calling, structured outputs, prompt engineering, multi-agent workflows, LLM evaluation and observability

Cloud & DevOps: AWS (Lambda, Bedrock, API Gateway, DynamoDB, S3, CloudFront, ECR), Docker, Terraform (IaC), GitHub Actions CI/CD (OIDC), serverless and event-driven architecture

Generative AI & ML: vector databases, embedding models, Hugging Face, scikit-learn, MediaPipe

Backend, Frontend & Databases: FastAPI (REST APIs, microservices), Next.js, React, Node.js, PostgreSQL, MySQL, DynamoDB

CERTIFICATIONS

Proficient AI Engineer – Ed Donner / The AI Engineer track (2026) ([credential](#)) | **Introduction to Machine Learning** – NPTEL / IIT Madras (2023) ([certificate](#))